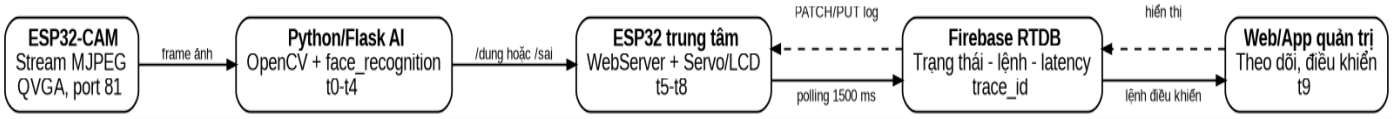
ĐO LƯỜNG VÀ PHÂN TÍCH ĐỘ TRỄ END-TO-END TRONG HỆ THỐNG CỔNG THÔNG MINH ỨNG DỤNG NHẬN DIỆN KHUÔN MẶT

Nguyễn Trần Khánh Nguyên, Phạm Văn Nhiệm, Võ Trường Quốc Quang, Nguyễn Quang Quân, Phạm Đắc Vinh
Khoa Điện - Điện tử , Đại Học Công Nghệ Kỹ Thuật Thành Phố Hồ Chí Minh

Tóm tắt- Bài báo trình bày hệ thống cổng thông minh ứng dụng nhận diện khuôn mặt và tập trung vào bài toán đo độ trễ end-to-end. Dựa trên mã nguồn của hệ thống, mô hình gồm ESP32-CAM truyền luồng ảnh MJPEG, Python/Flask xử lý nhận diện khuôn mặt, ESP32 trung tâm điều khiển servo, LCD, LED, buzzer và Firebase Realtime Database lưu lệnh, trạng thái cùng mốc thời gian. Chuỗi mốc t0 đến t9 được dùng để truy vết một thao tác từ lúc nhận ảnh, xử lý AI, gửi lệnh, ESP32 thực thi, ghi Firebase đến khi Web/App hiển thị kết quả. Kết quả thử nghiệm ghi nhận tổng độ trễ 3764 ms, trong đó độ trễ lớn nhất nằm ở cập nhật Web/App và truyền lệnh qua cơ chế polling Firebase.

Từ khóa- đo độ trễ end-to-end, Smart Gate, ESP32-CAM, ESP32, nhận diện khuôn mặt, Firebase, Web Dashboard, App quản trị.



Hình 1. Kiến trúc hệ thống và luồng đo độ trễ end-to-end của Smart Gate AI.

1. GIỚI THIỆU

Trong các hệ thống cổng thông minh, thời gian phản hồi là yếu tố quan trọng vì người dùng cần thấy cổng mở hoặc cảnh báo xuất hiện trong thời gian hợp lý. Nếu chỉ kiểm tra cổng có mở hay không, ta chưa đánh giá được chất lượng vận hành của toàn hệ thống. Do đó, đề tài này chọn hướng phân tích độ trễ end-to-end làm trọng tâm nghiên cứu.

Hệ thống được thiết kế theo mô hình nhiều khối: ESP32-CAM thu nhận hình ảnh, Python/Flask nhận diện khuôn mặt, ESP32 trung tâm điều khiển phần cứng và Firebase đồng bộ dữ liệu lên Web/App. Mỗi khối đều có thể tạo ra độ trễ riêng, vì vậy bài báo sử dụng chuỗi mốc thời gian để tách nhỏ tổng độ trễ và xác định điểm nghẽn.

2. MỤC TIÊU NGHIÊN CỨU

Mục tiêu chính là xây dựng một phương pháp đo và phân tích độ trễ cho mô hình cổng thông minh nhận diện khuôn mặt. Bên cạnh đó, hệ thống vẫn phải đáp ứng các chức năng cơ bản: nhận diện đúng thì mở cổng, nhận diện sai thì cảnh báo, hiển thị trạng thái trên LCD và đồng bộ dữ liệu lên Web/App quản trị.

Điểm đóng góp của bài báo là gắn trace_id và các mốc t0-t9 vào từng thao tác. Nhờ đó, hệ thống không chỉ chạy được mà còn có dữ liệu định lượng để đánh giá giai đoạn nào đang gây trễ nhiều nhất.

3. PHÂN TÍCH MÃ NGUỒN HỆ THỐNG

3.1. Khối ESP32-CAM

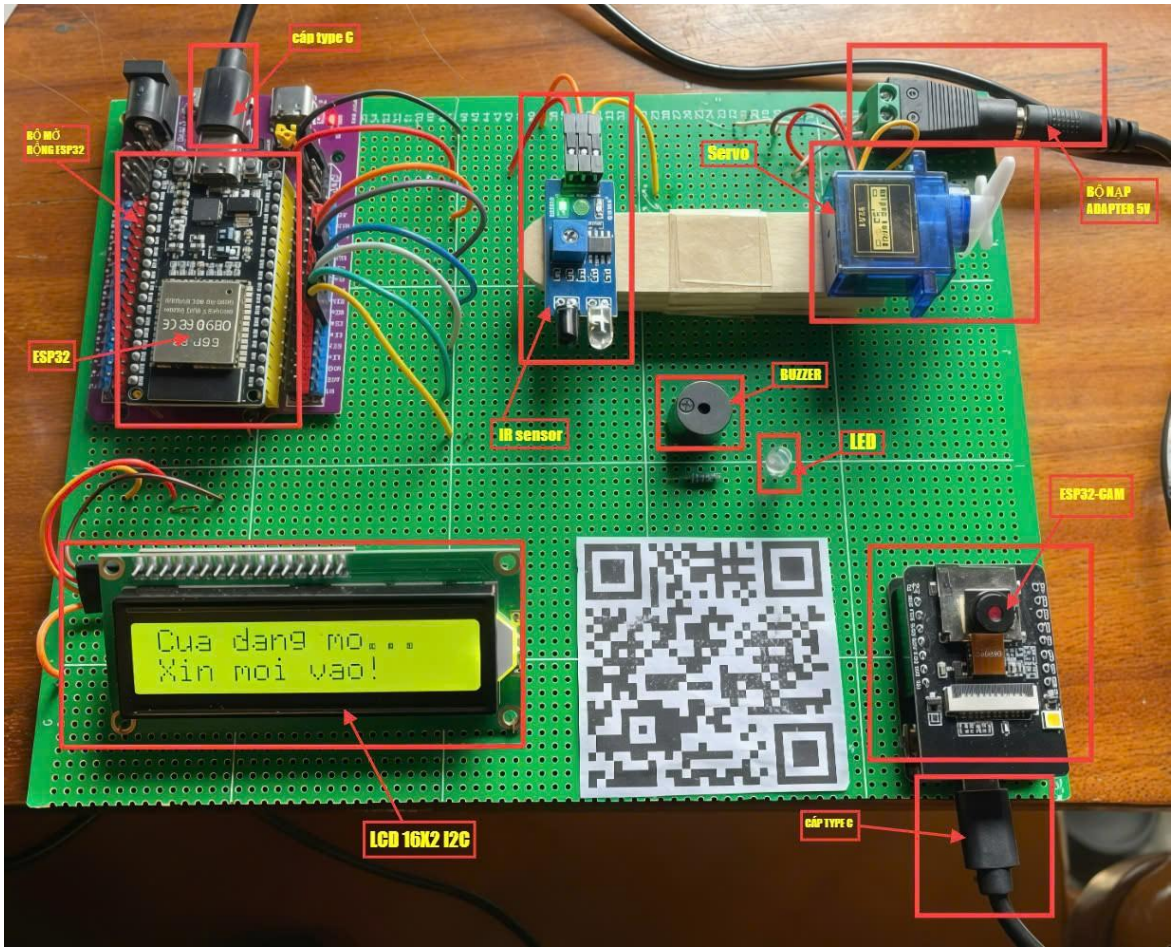
Mã nguồn ESP32_CAM.ino cấu hình camera AI Thinker, phát luồng MJPEG qua HTTP ở cổng 81. Camera được đặt ở độ phân giải QVGA 320x240, jpeg_quality bằng 15, fb_count bằng 2 và chế độ CAMERA_GRAB_LATEST. Các lựa chọn này phù hợp với mục tiêu giảm tải đường truyền và giữ frame mới nhất cho khối AI.

3.2. Khối Python/Flask AI

Trong app.py, chương trình dùng Flask, OpenCV, requests, queue và face_recognition. Hàm camera_worker nhận stream, tách JPEG, giải mã frame và ghi mốc t0, t1. Hàm ai_worker resize frame, chuyển BGR sang RGB, dùng HOG để phát hiện khuôn mặt và so khớp với ảnh mẫu chủ nhà ở tolerance 0,55. Khi phát hiện chủ nhà, chương trình gửi /dung; khi phát hiện người lạ, chương trình gửi /sai. Cooldown 3 giây cho /dung và 5 giây cho /sai giúp tránh gửi lệnh lặp liên tục.

3.3. Khối ESP32 trung tâm

Mã nguồn MAIN_ESP.ino cấu hình servo ở GPIO12, LED GPIO26, buzzer GPIO27, cảm biến IR GPIO13 và LCD I2C SDA25/SCL33. ESP32 chạy WebServer cục bộ với hai endpoint /dung và /sai, đồng thời đọc lệnh từ Firebase theo chu kỳ 1500 ms. Hàm openGate(), closeGate(), triggerAlarm() và checkFirebaseCommand() là các điểm xử lý chính. Khi có thao tác, ESP32 ghi các mốc t5, t6, t7, t8 và gửi bản ghi latency lên Firebase theo trace_id.



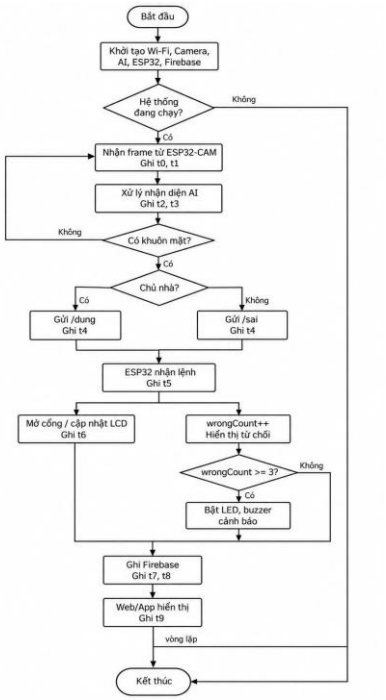
Hình 2. Mô hình phần cứng thực nghiệm của hệ thống Smart Gate AI.

Khối	Vai trò trong đo trễ	Thông tin từ code	Nhận xét
ESP32-CAM	Nguồn ảnh đầu vào	MJPEG stream port 81, QVGA 320x240, grab latest	Tối ưu để giảm kích thước frame
Python/Flask AI	Tạo t0-t4	camera_worker, ai_worker, send_signal, tolerance 0,55	Là khối xử lý nhận diện
ESP32 trung tâm	Tạo t5-t8	endpoint /dung, /sai; polling Firebase 1500 ms	Polling là nguyên nhân trễ đáng chú ý
Firebase + Web/App	Lưu và hiển thị t9	node smart_gate/latency và trang_thai_thuc	Dùng để truy vết thao tác

Bảng 1. Liên hệ giữa mã nguồn và vai trò của từng khối trong phép đo độ trễ.

4. LƯU ĐỒ THUẬT TOÁN

Lưu đồ ở Hình 3 được xây dựng theo phong cách thuật toán trong ngôn ngữ C: có khởi tạo, vòng lặp while, các khối xử lý tuần tự, khối điều kiện if/else và một điểm kết thúc chung. Cách vẽ này phản ánh đúng bản chất chương trình vì hệ thống hoạt động liên tục theo vòng lặp, không kết thúc sau một lần mở cổng.



Hình 3. Lưu đồ thuật toán chính của hệ thống

5. GIAO DIỆN GIÁM SÁT

Web Dashboard hiển thị luồng nhận diện và trạng thái hệ thống. Ứng dụng quản trị bổ sung màn hình đăng nhập, tạo nền tảng để mở rộng các chức năng như lịch sử truy cập, thống kê cảnh báo và điều khiển từ xa. Cả hai giao diện đều lấy dữ liệu thông qua Firebase nên phù hợp với mô hình giám sát thời gian thực.



Hình 4. Giao diện Web Dashboard và App quản trị của hệ thống.

6. PHƯƠNG PHÁP ĐO ĐỘ TRỄ

Mỗi thao tác được gắn trace_id để ghép dữ liệu giữa Python, ESP32, Firebase và Web/App. Các mốc t0-t9 được ghi trong quá trình xử lý. Với mã ESP32-CAM hiện tại, tcap có thể xem là mốc mở rộng khi camera gắn thêm thời điểm chụp vào header stream; còn bộ mốc ổn định dùng cho phân tích chính là t0 đến t9.

(1) $L_{total} = t_9 - t_0$
(2) $L_{cam} = t_0 - t_{cap}$
(3) $L_{decode} = t_1 - t_0$
(4) $L_{AI} = t_3 - t_2$
(5) $L_{cmd} = t_5 - t_4$
(6) $L_{ESP} = t_6 - t_5$
(7) $L_{FB} = t_8 - t_7$
(8) $L_{Web} = t_9 - t_8$
(9) $P_i = \frac{L_i}{L_{total}} \times 100\%$

Ghi chú: nếu ESP32-CAM chưa gắn mốc tcap thì bắt đầu phân tích từ t0.
Các mốc t0-t9 được truyền theo trace_id để ghép dữ liệu giữa Python, ESP32, Firebase và Web/App.

Hình 5. Công thức tính các thành phần độ trễ trong hệ thống.



Hình 6. Chuỗi mốc thời gian dùng để truy vết độ trễ end-to-end.

Mốc	Ý nghĩa	Khối tạo mốc
t0	Python nhận frame hoặc thao tác bắt đầu	Python/Web
t1	Giải mã JPEG xong	Python/OpenCV
t2	Bắt đầu xử lý nhận diện	Python AI
t3	AI xử lý xong	Python AI
t4	Gửi lệnh /dung, /sai hoặc lệnh Web/App	Python/Web
t5	ESP32 nhận lệnh	ESP32
t6	ESP32 bắt đầu thực thi	ESP32
t7	Bắt đầu ghi Firebase	ESP32
t8	Firebase phản hồi ghi xong	ESP32
t9	Web/App nhận và hiển thị kết quả	Web/App

Bảng 2. Ý nghĩa các mốc thời gian t0-t9 trong phép đo.

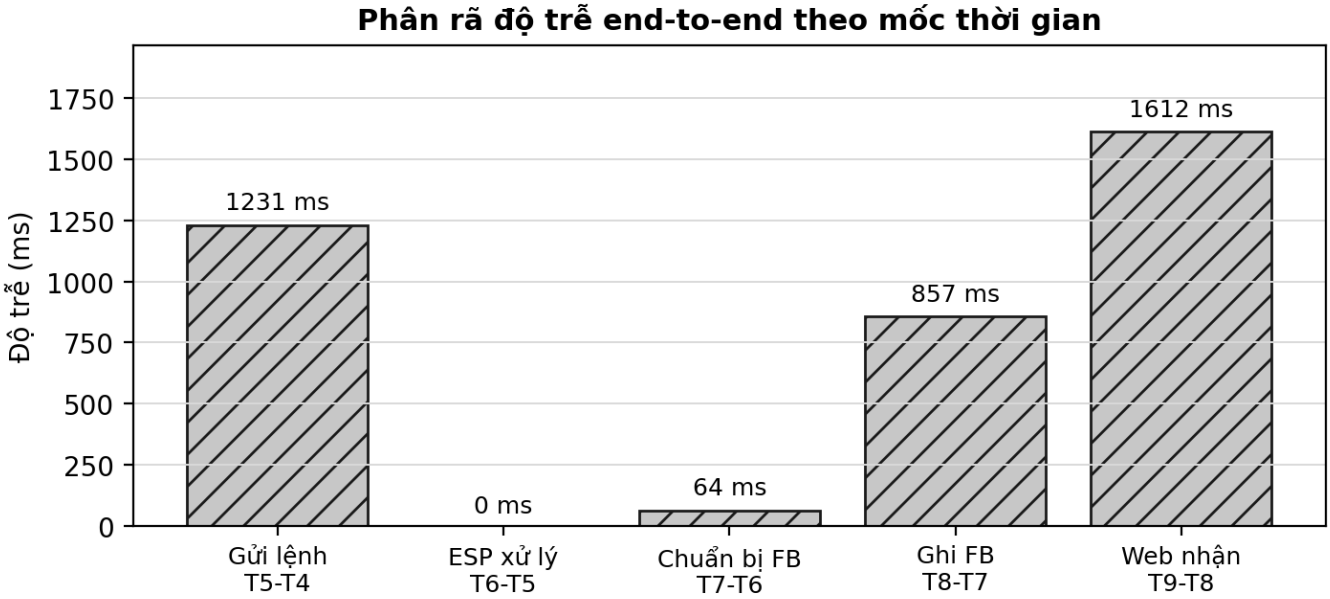
7. KẾT QUẢ VÀ PHÂN TÍCH

Một mẫu đo ghi nhận tổng độ trễ end-to-end là 3764 ms. Trong đó, giai đoạn gửi lệnh đến ESP32 mất 1231 ms, ghi Firebase mất 857 ms và Web/App cập nhật kết quả mất 1612 ms. Ngược lại, đoạn ESP32 nhận lệnh đến khi bắt đầu thực thi gần như bằng 0 ms, cho thấy bản thân vi điều khiển không phải nút thắt chính.

Kết quả này phù hợp với cấu trúc chương trình. ESP32 đọc lệnh từ Firebase theo chu kỳ 1500 ms nên lệnh có thể phải chờ đến lần polling tiếp theo. Ngoài ra, Web/App cũng cần thời gian nhận dữ liệu mới từ Firebase và cập nhật giao diện. Vì vậy, hướng tối ưu cần tập trung vào cơ chế đồng bộ dữ liệu thay vì chỉ thay đổi phần điều khiển servo.



Hình 7. Màn hình ghi nhận các mốc thời gian của thao tác điều khiển.



Hình 8. Biểu đồ phân rã độ trễ theo từng giai đoạn.

Giai đoạn	Công thức	Giá trị	Tỷ lệ	Nhận xét
Gửi lệnh đến ESP32	t5 - t4	1231 ms	32,7%	Bị ảnh hưởng bởi mạng và polling
ESP32 thực thi	t6 - t5	0 ms	0,0%	Phản hồi gần như tức thời
Chuẩn bị ghi Firebase	t7 - t6	64 ms	1,7%	Chi phí nội bộ nhỏ
Ghi Firebase	t8 - t7	857 ms	22,8%	Phụ thuộc kết nối cloud
Web/App cập nhật	t9 - t8	1612 ms	42,8%	Thành phần chiếm tỷ trọng lớn nhất
Tổng	t9 - t0	3764 ms	100%	Độ trễ người dùng cảm nhận

Bảng 3. Phân rã độ trễ end-to-end theo dữ liệu thực nghiệm.

8. THẢO LUẬN

Về chức năng, hệ thống đáp ứng được yêu cầu đề tài: nhận diện đúng thì mở cổng, nhận diện sai thì tăng bộ đếm và cảnh báo, trạng thái được hiển thị tại LCD và đồng bộ lên giao diện. Về hiệu năng, phương pháp đo t0-t9 giúp chỉ ra rõ điểm nghẽn nằm ở đường truyền dữ liệu và giao diện chứ không nằm ở thao tác điều khiển phần cứng.

Nếu triển khai ở quy mô thực tế, hệ thống cần tăng số mẫu đo để tính trung bình, trung vị, độ lệch chuẩn và giá trị lớn nhất. Ngoài ra, cần giảm chu kỳ polling hoặc chuyển sang cơ chế push/event để rút ngắn độ trễ từ Web/App đến ESP32.

9. KẾT LUẬN

Bài báo đã trình bày một mô hình cổng thông minh ứng dụng nhận diện khuôn mặt với trọng tâm là đo và phân tích độ trễ end-to-end. Từ việc đọc và phân tích mã nguồn, các mốc thời gian đã được liên hệ trực tiếp với từng khối xử lý trong hệ thống. Kết quả thử nghiệm cho thấy tổng độ trễ của một thao tác là 3764 ms, trong đó Web/App cập nhật và cơ chế truyền lệnh là các thành phần cần tối ưu nhất.

10. HƯỚNG PHÁT TRIỂN

- Bổ sung header thời điểm chụp trên ESP32-CAM để đo chính xác thêm thành phần L_cam.

- Giảm độ trễ bằng cơ chế push/event thay cho polling Firebase 1500 ms.
- Thu thập nhiều mẫu đo hơn để xây dựng thống kê trung bình, trung vị và độ lệch chuẩn.
- Tăng bảo mật bằng rule Firebase, tài khoản quản trị và ẩn hoàn toàn khóa bí mật trong mã nguồn.

TÀI LIỆU THAM KHẢO

[1] Espressif Systems, ESP32 Series Datasheet.
[2] Espressif Systems, ESP32-CAM Technical Reference.
[3] Google Firebase, Firebase Realtime Database Documentation.
[4] Pallets Projects, Flask Documentation.
[5] Adam Geitgey, face_recognition Python Library.